

SUPERSEDES ARCHITECTURE REPORT V1.0

# Final Technical Decisions Report

Bloom Parfums — Stateless-Customer E-Commerce Platform

Next.js 14 Hybrid

Laravel API-only

MySQL + Redis

Cookie Wishlist

Admin-Only Auth

Guest Checkout

Prepared by: Principal Architect AI

Date: February 25, 2026

Version: 2.0 — FINAL

Status: Approved for Development

## Final Technical Decisions Report — Bloom Parfums

**Prepared by:** Principal Full-Stack Architect AI

**Date:** February 25, 2026

**Supersedes:** ARCHITECTURE\_REPORT.md v1.0

**Status:** FINAL — Ready for Development

*This document finalizes all architectural decisions for Bloom Parfums under one non-negotiable constraint set: **no public user auth system, cookie-based wishlist only, admin-only backend access control.** Every decision below is final and justified for production.*

### Table of Contents

1. Architecture Validation
2. Chosen Tech Stack & Rationale
3. Wishlist Cookie System Design
4. Admin Authentication Strategy
5. Optimized API Architecture
6. Production Readiness Notes
7. Final Verdict

# 1. Architecture Validation

## 1.1 What the Original Report Got Wrong

The original `ARCHITECTURE_REPORT.md` was designed around a **customer-authenticated e-commerce platform** — a valid pattern, but the wrong one for this product. The following components must be **cut entirely**:

| Original Proposal                                                               | Decision        | Reason                                                                                        |
|---------------------------------------------------------------------------------|-----------------|-----------------------------------------------------------------------------------------------|
| <code>users</code> table                                                        | <b>REMOVE</b>   | No public user registration or login exists                                                   |
| <code>addresses</code> table (user-owned)                                       | <b>REMOVE</b>   | Address is collected at checkout time only — snapshot into order                              |
| <code>wishlist_items</code> table                                               | <b>REMOVE</b>   | Wishlist lives in browser cookie, never persists to DB                                        |
| User Laravel Model                                                              | <b>REMOVE</b>   | Replace with <code>Admin</code> model only                                                    |
| <code>LoginRequest</code> / <code>RegisterRequest</code> (public)               | <b>REMOVE</b>   | No public auth endpoints                                                                      |
| <code>OrderPolicy</code> (user ownership)                                       | <b>SIMPLIFY</b> | Guest orders identified by <code>order_number</code> + <code>phone</code> lookup, not session |
| <code>WishlistPolicy</code>                                                     | <b>REMOVE</b>   | Wishlist is client-side only                                                                  |
| <code>authService.login()</code> in Next.js                                     | <b>REMOVE</b>   | Public login route does not exist                                                             |
| <code>lib/auth.ts</code> client helpers                                         | <b>REMOVE</b>   | Replaced by admin-only middleware                                                             |
| <code>/login</code> , <code>/register</code> , <code>/dashboard</code> (public) | <b>REMOVE</b>   | Exposed public auth surface that no longer exists                                             |
| JWT refresh interceptor in <code>api.ts</code>                                  | <b>SIMPLIFY</b> | Only admin uses tokens; complexity is not justified for a single user                         |
| <code>loyalty_points</code> on User                                             | <b>REMOVE</b>   | Depends on user accounts                                                                      |

## 1.2 What the Original Report Got Right

| Original Proposal                               | Decision    | Reason                                          |
|-------------------------------------------------|-------------|-------------------------------------------------|
| Product, Brand, Category, ShippingMethod models | <b>KEEP</b> | Core commerce entities, unchanged               |
| Order, OrderItem, OrderStatusHistory tables     | <b>KEEP</b> | Required for checkout and tracking              |
| Coupon table                                    | <b>KEEP</b> | Discount system is backend-validated            |
| Review, ProductImage tables                     | <b>KEEP</b> | Product content and social proof                |
| Service layer (CartService, OrderService)       | <b>KEEP</b> | Business logic belongs here, not in controllers |
| API Resources for response shaping              | <b>KEEP</b> | Prevents field over-exposure, correct pattern   |
| Redis caching for products/brands/categories    | <b>KEEP</b> | Essential for public-facing catalog performance |
| Laravel FormRequest validation                  | <b>KEEP</b> | Cleanest validation strategy in Laravel         |
| <code>ForceJsonResponse</code> middleware       | <b>KEEP</b> | Mandatory for API-only backend                  |

## 1.3 Identified Overengineering in Original

- **JWT refresh loop** — A single admin token does not need a refresh interceptor. A 24-hour HttpOnly session cookie is sufficient.
- **`PaginatedResponse` type on Wishlist** — The wishlist is never fetched from the backend. This type only applies to product/order listings.
- **`serverFetch<User>` in DashboardPage** — The public dashboard pattern is entirely removed. No server-side user pre-fetch needed.
- **`OrderPolicy` ownership check** — Without user accounts, orders are looked up by `order_number` + customer `phone` combination (public tracking). No policy needed.
- **Guest checkout `user_id: nullable`** — All orders are guest orders. `user_id` column is dropped entirely.

## 2. Chosen Tech Stack & Rationale

### 2.1 Frontend — Next.js

| Concern                            | Decision                                     | Rationale                                                                                               |
|------------------------------------|----------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Homepage ( / )                     | <b>SSG + ISR (revalidate: 3600)</b>          | Marketing content changes infrequently. Build-time render + hourly revalidation = zero TTFB for users   |
| Collection page ( /collection )    | <b>CSR with SWR</b>                          | Filter state changes frequently per user interaction; SSR would thrash on every filter change           |
| Product detail ( /product/[slug] ) | <b>ISR (revalidate: 1800)</b>                | Product data changes rarely; pre-render known slugs, fallback for new products                          |
| Checkout ( /checkout )             | <b>CSR</b>                                   | Fully interactive form, no SEO value, no benefit from server render                                     |
| Order Status ( /order-status )     | <b>CSR + SWR polling (30s)</b>               | Must reflect real-time backend changes without WebSocket overhead                                       |
| Admin panel ( /admin/* )           | <b>CSR behind Next.js Middleware</b>         | No SEO needed; Middleware blocks access before any component renders                                    |
| API communication                  | <b>SWR for reads, Axios for mutations</b>    | SWR handles cache, deduplication, revalidation; Axios gives fine-grained control over POST/PATCH/DELETE |
| State management                   | <b>Zustand (cart) + cookies (wishlist)</b>   | Cart needs reactive cross-component state; Wishlist needs zero-backend persistence                      |
| Admin auth state                   | <b>Single HttpOnly cookie + Zustand flag</b> | Cookie holds the token server-side securely; Zustand flag drives UI conditional rendering               |

#### What is NOT built on the frontend:

- No /login page (public)
- No /register page
- No /dashboard (customer profile)
- No user session management

## 2.2 Backend — Laravel

| Concern              | Decision                                                             | Rationale                                                                             |
|----------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Auth method          | <b>Laravel Sanctum — token-based, single admin</b>                   | Minimal footprint, built-in, no JWT dependency                                        |
| Admin token storage  | <b>HttpOnly, Secure, SameSite=Strict cookie</b>                      | Immune to XSS; enforced by browser                                                    |
| Controller structure | <b>One controller per resource, thin</b>                             | Delegates all logic to Service layer                                                  |
| Service layer        | <b>Yes — CartService, OrderService, CouponService, ReviewService</b> | Business logic must not live in controllers or models                                 |
| Validation           | <b>FormRequest per mutation</b>                                      | Centralized, typed, reusable. Fails fast before any business logic runs               |
| API versioning       | <b>/api/v1/ prefix, no breaking change policy for v1</b>             | Simple prefix is sufficient; versioning complexity (content negotiation) is premature |
| Response envelope    | <b>{ data, message, meta? }</b>                                      | Consistent contract; frontend never parses raw Eloquent output                        |
| Guest order lookup   | <b>order_number + phone match</b>                                    | No session needed. Stateless lookup. Secure enough for order status tracking          |

## 2.3 Database — Final Entity Set

### Entities that REMAIN:

| Table                               | Justification                        |
|-------------------------------------|--------------------------------------|
| <code>admins</code>                 | Single-row admin credential table    |
| <code>brands</code>                 | Core filter + product attribute      |
| <code>categories</code>             | Core filter + product classification |
| <code>products</code>               | Core sellable entity                 |
| <code>product_images</code>         | Gallery support                      |
| <code>shipping_methods</code>       | Checkout delivery options            |
| <code>coupons</code>                | Server-validated discounts           |
| <code>orders</code>                 | Guest order creation                 |
| <code>order_items</code>            | Line items per order                 |
| <code>order_status_histories</code> | Timeline data for tracking page      |
| <code>reviews</code>                | Post-purchase ratings                |

**Entities that are DROPPED:**

| Dropped Table                           | Reason                                           |
|-----------------------------------------|--------------------------------------------------|
| <code>users</code>                      | No customer accounts                             |
| <code>addresses</code>                  | Absorbed as JSON snapshot on <code>orders</code> |
| <code>wishlist_items</code>             | Cookie-only, never persisted                     |
| <code>password_reset_tokens</code>      | No public auth                                   |
| <code>sessions</code> (Laravel default) | Admin uses token, not session driver             |

## 3. Wishlist Cookie System Design

### 3.1 Decision: Cookies Over Every Alternative

| Option                      | Verdict         | Reason                                                                                                                    |
|-----------------------------|-----------------|---------------------------------------------------------------------------------------------------------------------------|
| Database (user-linked)      | <b>Rejected</b> | No user accounts exist                                                                                                    |
| <code>localStorage</code>   | <b>Rejected</b> | Not readable by Next.js Middleware or Server Components; cleared unexpectedly in private mode                             |
| <code>sessionStorage</code> | <b>Rejected</b> | Lost on tab close — unusable for 30-day persistence                                                                       |
| Server session (Laravel)    | <b>Rejected</b> | Requires backend roundtrip per page load for anonymous state; introduces scaling complexity                               |
| <b>Client cookie</b>        | <b>Chosen</b>   | Universal read access in browser AND Next.js Server Components/Middleware; 30-day expiry is native; no backend dependency |

### 3.2 Cookie Specification

```
Name:      bloom_wishlist
Value:     JSON array of product IDs - e.g. [12, 47, 103]
Expires:   30 days from last write (refreshed on every add/remove)
Path:      /
SameSite:  Lax
Secure:    true (production)
HttpOnly:  false ← Required - Next.js client code must read/write it
Max-Age:   2592000 (30 × 24 × 60 × 60 seconds)
```

**Why `HttpOnly: false` :**

The wishlist cookie is not a credential. It contains only non-sensitive product IDs. JavaScript must read and write it for the wishlist UI to function without a server roundtrip. `HttpOnly` is reserved for auth tokens only.

### 3.3 Data Structure

```
// Type: number[]
// Stored as: JSON.stringify([12, 47, 103])
// Max items enforced in code: 50 products
// Max cookie size: ~250 bytes for 50 IDs — well within 4KB browser limit

const WISHLIST_COOKIE = 'bloom_wishlist';
const MAX_ITEMS       = 50;
const TTL_DAYS        = 30;
```

#### Why IDs only, no product data:

- Keeps cookie size minimal
- Product data (name, price, image) is fetched from the API at read time
- Price changes are always reflected — no stale data risk

### 3.4 Client-Side Operations

```
// Read wishlist
function getWishlist(): number[] {
  const raw = Cookies.get(WISHLIST_COOKIE);
  try { return raw ? JSON.parse(raw) : []; }
  catch { return []; }
}

// Add to wishlist
function addToWishlist(productId: number): void {
  const ids = getWishlist();
  if (ids.includes(productId) || ids.length ≥ MAX_ITEMS) return;
  const next = [...ids, productId];
  Cookies.set(WISHLIST_COOKIE, JSON.stringify(next), {
    expires: TTL_DAYS,
    sameSite: 'lax',
    secure: process.env.NODE_ENV === 'production',
  });
}

// Remove from wishlist
function removeFromWishlist(productId: number): void {
  const next = getWishlist().filter(id ⇒ id !== productId);
  Cookies.set(WISHLIST_COOKIE, JSON.stringify(next), {
    expires: TTL_DAYS,
    sameSite: 'lax',
    secure: process.env.NODE_ENV === 'production',
  });
}
```

### 3.5 Backend Sync — Product Validation Only

The backend is **never written to** for wishlist operations. Its only role is to validate that products in the cookie still exist and are active when the wishlist page renders.

```
GET /api/v1/products/validate?ids=12,47,103

Response:
{
  "data": [
    { "id": 12, "active": true, "name": "SUGAR POP", "price": 140, "image_url": "... " },
    { "id": 47, "active": false, "name": null, "price": null, "image_url": null },
    { "id": 103, "active": true, "name": "OVER DOSE", "price": 100, "image_url": "... " }
  ]
}
```

The frontend then:

- 1. Removes IDs where `active: false` from the cookie silently.
- 2. Renders only active products.

3.6 Edge Case Handling

| Edge Case                    | Behavior                                                                                                                     |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Product deleted from catalog | <code>validate</code> endpoint returns <code>active: false</code> → silently removed from cookie on next wishlist page visit |
| Cookie cleared by user       | Wishlist is empty — no recovery, no error. Expected behavior                                                                 |
| Cookie expired after 30 days | Browser deletes cookie automatically — wishlist resets. No action needed                                                     |
| User adds >50 items          | Client enforces <code>MAX_ITEMS = 50</code> — silently rejects add                                                           |
| Malformed cookie value       | <code>try/catch</code> in <code>getWishlist()</code> returns empty array and rewrites a clean cookie                         |
| Product price changed        | Price always fetched from API at render time — always current                                                                |

4. Admin Authentication Strategy

4.1 Architecture Decision

One admin. One token. Stored in an `HttpOnly` cookie. Validated server-side on every admin request.

No user roles, no role middleware hierarchies, no multi-admin permission tables. Create a single `admins` table with one row.



## 4.2 Database

```
CREATE TABLE admins (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  email       VARCHAR(191) NOT NULL UNIQUE,
  password    VARCHAR(255) NOT NULL,
  created_at  TIMESTAMP     NULL DEFAULT NULL,
  updated_at  TIMESTAMP     NULL DEFAULT NULL
);
```

No `role` column. No `permissions` table. This application has one admin. YAGNI.

## 4.3 Laravel Backend

Single route group, single middleware:

```
// routes/api.php

// Public routes - no auth
Route::prefix('v1')->group(function () {
    Route::get('/products', [ProductController::class, 'index']);
    Route::get('/products/{slug}', [ProductController::class, 'show']);
    Route::get('/products/validate', [ProductController::class, 'validate']);
    Route::get('/brands', [BrandController::class, 'index']);
    Route::get('/categories', [CategoryController::class, 'index']);
    Route::get('/shipping-methods', [ShippingMethodController::class, 'index']);
    Route::post('/orders', [OrderController::class, 'store']);
    Route::get('/orders/{number}/track', [OrderController::class, 'track']);
    Route::post('/reviews', [ReviewController::class, 'store']);
    Route::get('/products/{slug}/reviews', [ReviewController::class, 'index']);
    Route::post('/cart/coupon/validate', [CouponController::class, 'validate']);
});

// Admin routes - Sanctum token required
Route::prefix('v1/admin')->middleware(['auth:sanctum', 'admin'])->group(function () {
    Route::post('/auth/login', [AdminAuthController::class, 'login']);
    Route::post('/auth/logout', [AdminAuthController::class, 'logout']);
    Route::get('/auth/me', [AdminAuthController::class, 'me']);

    Route::apiResource('products', Admin\ProductController::class);
    Route::apiResource('brands', Admin\BrandController::class);
    Route::apiResource('categories', Admin\CategoryController::class);
    Route::apiResource('coupons', Admin\CouponController::class);
    Route::get('/orders', [Admin\OrderController::class, 'index']);
    Route::patch('/orders/{id}/status', [Admin\OrderController::class, 'updateStatus']);
    Route::get('/reviews', [Admin\ReviewController::class, 'index']);
    Route::patch('/reviews/{id}/approve', [Admin\ReviewController::class, 'approve']);
});
```

**EnsureAdmin middleware** — Guards against token belonging to a non-admin model (belt-and-suspenders):

```
class EnsureAdmin {
  public function handle(Request $request, Closure $next): Response {
    if (!$request->user() instanceof Admin) {
      return response()->json(['message' => 'Forbidden'], 403);
    }
    return $next($request);
  }
}
```

#### 4.4 Admin Token — Cookie Transport

The Sanctum token is **NOT returned to the JavaScript layer**. It is set as a `Set-Cookie` header by Laravel on login and cleared on logout:

```
// AdminAuthController@login
public function login(LoginRequest $request): JsonResponse {
  $admin = Admin::where('email', $request->email)->first();

  if (!$admin || !Hash::check($request->password, $admin->password)) {
    return response()->json(['message' => 'Unauthorized'], 401);
  }

  $token = $admin->createToken('admin-session')->plainTextToken;

  return response()->json(['message' => 'Authenticated'])
    ->withCookie(cookie(
      name:      'admin_token',
      value:     $token,
      minutes:   1440,           // 24 hours
      path:      '/',
      secure:    true,
      httpOnly: true,
      sameSite: 'Strict',
    ));
}
```

#### 4.5 Next.js Frontend — Admin Access Control

All `/admin/*` routes are protected by **Next.js Middleware** — the request is intercepted before any React component renders:

```
// middleware.ts
import { NextRequest, NextResponse } from 'next/server';

export function middleware(request: NextRequest) {
  const isAdminRoute = request.nextUrl.pathname.startsWith('/admin');
  const adminToken = request.cookies.get('admin_token');

  if (isAdminRoute && !adminToken) {
    return NextResponse.redirect(new URL('/admin/login', request.url));
  }
  return NextResponse.next();
}

export const config = {
  matcher: ['/admin/:path*'],
};
```

The admin login page ( `/admin/login` ) is the only unprotected admin route. It posts credentials to `POST /api/v1/admin/auth/login`, which sets the HttpOnly cookie. No token is ever visible in JavaScript.

4.6 Why This Is Safer Than the Original Proposal

| Original                                            | Final Decision                                                            | Security Gain                                          |
|-----------------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------|
| JWT in <code>js-cookie</code> (JavaScript-readable) | HttpOnly cookie (JS-blind)                                                | Eliminates XSS token theft entirely                    |
| Public <code>/login</code> route for customers      | No public auth routes at all                                              | Zero public auth attack surface                        |
| JWT refresh interceptor in Axios                    | Stateless Sanctum token; on expiry, redirect to <code>/admin/login</code> | Simpler, no refresh race conditions                    |
| <code>User</code> model with <code>role</code> enum | Separate <code>Admin</code> model                                         | Cannot escalate a customer to admin — they don't exist |

5. Optimized API Architecture

5.1 Public Endpoint Summary

All endpoints below require **no authentication**. They are the public contract of the storefront.

| Method | Endpoint                        | Description                    | Cache        |
|--------|---------------------------------|--------------------------------|--------------|
| GET    | /api/v1/products                | Paginated catalog with filters | SWR 5 min    |
| GET    | /api/v1/products/{slug}         | Product detail                 | ISR 30 min   |
| GET    | /api/v1/products/validate       | Wishlist cookie validation     | No cache     |
| GET    | /api/v1/products/featured       | BestSellers section            | Redis 1h     |
| GET    | /api/v1/brands                  | Brand list for filters         | Redis 6h     |
| GET    | /api/v1/categories              | Category tree                  | Redis 6h     |
| GET    | /api/v1/shipping-methods        | Delivery options               | Redis 24h    |
| POST   | /api/v1/orders                  | Create guest order from cart   | No cache     |
| GET    | /api/v1/orders/{number}/track   | Guest order tracking           | No cache     |
| POST   | /api/v1/reviews                 | Submit product review          | No cache     |
| GET    | /api/v1/products/{slug}/reviews | Approved reviews for product   | Redis 15 min |
| POST   | /api/v1/cart/coupon/validate    | Validate coupon code           | No cache     |

## 5.2 Admin Endpoint Summary

All require `admin_token` HttpOnly cookie (Sanctum).

| Method              | Endpoint                           | Description                      |
|---------------------|------------------------------------|----------------------------------|
| POST                | /api/v1/admin/auth/login           | Admin login (sets cookie)        |
| POST                | /api/v1/admin/auth/logout          | Admin logout (clears cookie)     |
| GET/POST/PUT/DELETE | /api/v1/admin/products             | Product CRUD                     |
| GET/POST/PUT/DELETE | /api/v1/admin/brands               | Brand CRUD                       |
| GET/POST/PUT/DELETE | /api/v1/admin/categories           | Category CRUD                    |
| GET/POST/PATCH      | /api/v1/admin/coupons              | Coupon management                |
| GET                 | /api/v1/admin/orders               | All orders list                  |
| PATCH               | /api/v1/admin/orders/{id}/status   | Update order status              |
| GET                 | /api/v1/admin/reviews              | All reviews (pending + approved) |
| PATCH               | /api/v1/admin/reviews/{id}/approve | Approve/reject review            |

5.3 Eliminated Endpoints (From Original Report)

| Original Endpoint           | Removed Reason                           |
|-----------------------------|------------------------------------------|
| POST /auth/register         | No public registration                   |
| POST /auth/login (public)   | No public login                          |
| POST /auth/refresh          | HttpOnly cookie auth — no refresh needed |
| GET /auth/me (public)       | Does not exist                           |
| GET/POST/DELETE /wishlist   | Wishlist is cookie-only                  |
| GET/POST/PUT /addresses     | Address is a JSON field on order         |
| GET /orders (customer list) | No customer accounts                     |

5.4 Order Tracking — Guest Design

Since no user accounts exist, orders are tracked by a public lookup:

```
GET /api/v1/orders/{order_number}/track
```

**This endpoint requires no authentication.** It returns the order status history and basic shipment info. To prevent enumeration attacks:

- `order_number` format must be complex enough (e.g. `LX-8921-Q7F`) — 10-character alphanumeric.
- Rate-limited to **10 requests/minute per IP** to prevent brute-force lookup.
- Returns identical `404` response whether order does not exist or is simply not found — never leaks enumeration hints.

5.5 Guest Order Creation

The `POST /api/v1/orders` endpoint replaces checkout. The address is embedded directly:

```
Request:
{
  "first_name": "Ayoub",
  "last_name": "Laghzal",
  "phone": "+212611955060",
  "city": "Casablanca",
  "quartier": "Hay Hassani",
  "zip_code": "20230",
  "address_line": "N° 10, Rue XYZ, Appt 3",
  "shipping_method_id": 2,
  "coupon_code": "PROMO10",
  "items": [
    { "product_id": 12, "quantity": 1 },
    { "product_id": 47, "quantity": 3 }
  ]
}

Response 201:
{
  "data": {
    "order_number": "LX-8921-Q7F",
    "status": "confirmed",
    "total": 275.00,
    "shipping_cost": 35.00,
    "discount": 0.00,
    "estimated_delivery": "2026-02-28",
    "items": [ ... ]
  }
}
```

The `OrderService` is responsible for:

1. Validating all product IDs exist and have sufficient stock.
2. Resolving prices from DB — never trusting frontend-supplied prices.
3. Atomically decrementing stock.
4. Validating and applying coupon.
5. Persisting order + items + initial status history entry.
6. Dispatching `SendOrderConfirmationEmail` to queue.
7. Returning the response.

## 5.6 Rate Limiting Strategy

```
Public catalog endpoints: 120 req/min per IP
Order creation:           10 req/min per IP
Order tracking:           10 req/min per IP
Review submission:        5 req/min per IP
Coupon validation:        20 req/min per IP
Admin login:              5 req/min per IP (hard block after 10 failures/hour)
Admin endpoints (authenticated): 300 req/min per token
```

5.7 Error Response Normalization

Every error, without exception, returns this envelope:

```
{
  "message": "Human-readable error summary",
  "errors": {
    "field_name": ["Specific validation failure"]
  },
  "code": "MACHINE_READABLE_CODE"
}
```

Machine-readable codes enable the frontend to handle specific cases programmatically without parsing message strings:

| Code                | Meaning                              |
|---------------------|--------------------------------------|
| VALIDATION_ERROR    | 422 — FormRequest failure            |
| STOCK_INSUFFICIENT  | 409 — Cannot fulfill quantity        |
| COUPON_EXPIRED      | 410 — Coupon no longer valid         |
| COUPON_EXHAUSTED    | 410 — Usage limit reached            |
| ORDER_NOT_FOUND     | 404 — Tracking lookup failed         |
| PRODUCT_UNAVAILABLE | 410 — Product is inactive            |
| UNAUTHORIZED        | 401 — Admin token missing or invalid |

6. Production Readiness Notes

6.1 Security Headers

Set via Laravel middleware on every response:

```
'X-Content-Type-Options'    => 'nosniff',
'X-Frame-Options'          => 'DENY',
'Referrer-Policy'          => 'strict-origin-when-cross-origin',
'Permissions-Policy'       => 'camera=(), microphone=(), geolocation=()',
'Content-Security-Policy'   => "default-src 'self'; img-src * data:; font-src 'self'; script-src 'self';"
```

6.2 Cookie Security Checklist

| Cookie         | HttpOnly         | Secure       | SameSite | Expires  |
|----------------|------------------|--------------|----------|----------|
| admin_token    | ✔ Yes            | ✔ Yes        | Strict   | 24 hours |
| bloom_wishlist | ✗ No (by design) | ✔ Yes (prod) | Lax      | 30 days  |

`bloom_wishlist` **must not** be `HttpOnly` — the JavaScript wishlist logic writes to it. This is intentional and documented. It contains zero sensitive data.

### 6.3 Environment Separation

| Variable                              | local          | staging                 | production      |
|---------------------------------------|----------------|-------------------------|-----------------|
| <code>APP_ENV</code>                  | local          | staging                 | production      |
| <code>APP_DEBUG</code>                | true           | false                   | false           |
| <code>DB_HOST</code>                  | localhost      | private IP              | RDS/managed DB  |
| <code>CACHE_DRIVER</code>             | file           | redis                   | redis           |
| <code>QUEUE_CONNECTION</code>         | sync           | redis                   | redis           |
| <code>SANCTUM_STATEFUL_DOMAINS</code> | localhost:3000 | staging.bloomparfums.ma | bloomparfums.ma |
| <code>SESSION_SECURE_COOKIE</code>    | false          | true                    | true            |

On staging and production, `APP_DEBUG=false` is non-negotiable. Stack traces in API responses are a direct security leak.

### 6.4 Database Performance

- All foreign keys have indexes (defined in schema).
- `products` table: composite index on `(is_active, is_featured)` for BestSellers query.
- `orders` table: index on `order_number` for O(1) tracking lookup.
- `order_status_histories` : append-only, never updated — no performance concern.
- `reviews` table: partial index on `is_approved = true` for public review queries.

**No full-text search engine needed at this scale.** MySQL `LIKE` on `products.name` with a length guard ( `≥ 3 chars` ) is adequate for catalog search up to 50,000 products. Add Meilisearch only when query time exceeds 200ms in production.



## 6.5 Caching Strategy — Final

| Data              | Mechanism                                                    | TTL    | Invalidation                    |
|-------------------|--------------------------------------------------------------|--------|---------------------------------|
| Featured products | <code>Cache::remember('products.featured', 3600, ...)</code> | 1h     | On admin product update         |
| Brand list        | <code>Cache::remember('brands.all', 21600, ...)</code>       | 6h     | On admin brand update/create    |
| Category tree     | <code>Cache::remember('categories.tree', 21600, ...)</code>  | 6h     | On admin category update/create |
| Product detail    | <code>Cache::remember("product.{\$slug}", 1800, ...)</code>  | 30 min | On admin product update         |
| Approved reviews  | <code>Cache::remember("reviews.{\$slug}", 900, ...)</code>   | 15 min | On review approval              |

Use cache tags ( `Cache::tags(['products'])→flush()` ) to invalidate all product-related cache keys on any admin product mutation.

## 6.6 Queue Strategy

```
Connection: Redis (staging + production), sync (local)
Queues:
- default      : order confirmation emails
- media        : review image resizing
- low          : analytics events, cache warming

Retry attempts: 3
Retry delay:    exponential (60s, 120s, 240s)
Failed jobs:    stored in failed_jobs table for inspection
```

## 6.7 File Upload — Review Images

- Max file size: **5 MB** per image, **3 images** per review.
- Accepted MIME types: `image/jpeg`, `image/png`, `image/webp` — validated from file content (not extension).
- Storage: `Storage::disk('s3')` → bucket `bloom-parfums-media`.
- Processing: queued `ProcessReviewImage` job handles resize (800px wide max) and format conversion to WebP.
- Public URL: served via Cloudflare CDN in front of S3 — never expose raw S3 URLs to the frontend.

## 6.8 What NOT to Build (Final Decision)

The following items represent scope creep, premature optimization, or architectural dead ends for V1:

| Do NOT Build                          | Reason                                                                                            |
|---------------------------------------|---------------------------------------------------------------------------------------------------|
| Customer accounts / user auth         | Explicitly out of scope by system constraints                                                     |
| WebSockets / real-time order tracking | Polling every 30s via SWR is sufficient; WebSocket infra is 10× the complexity                    |
| Search engine (Meilisearch, Algolia)  | MySQL LIKE is sufficient to 50K products; revisit at scale                                        |
| Loyalty points system                 | Requires user accounts — does not exist                                                           |
| Social login (Google, Facebook)       | No public auth — irrelevant                                                                       |
| Multi-admin with roles/permissions    | One admin. Ship it. Add when headcount grows                                                      |
| Cart persistence on backend           | Client-managed cart (Zustand) is sent to <code>/orders</code> on checkout. No cart storage needed |
| GraphQL                               | REST is sufficient; GraphQL overhead is not justified for this data shape                         |
| Microservices                         | This is a single-store e-commerce app with one admin — monolith is correct                        |

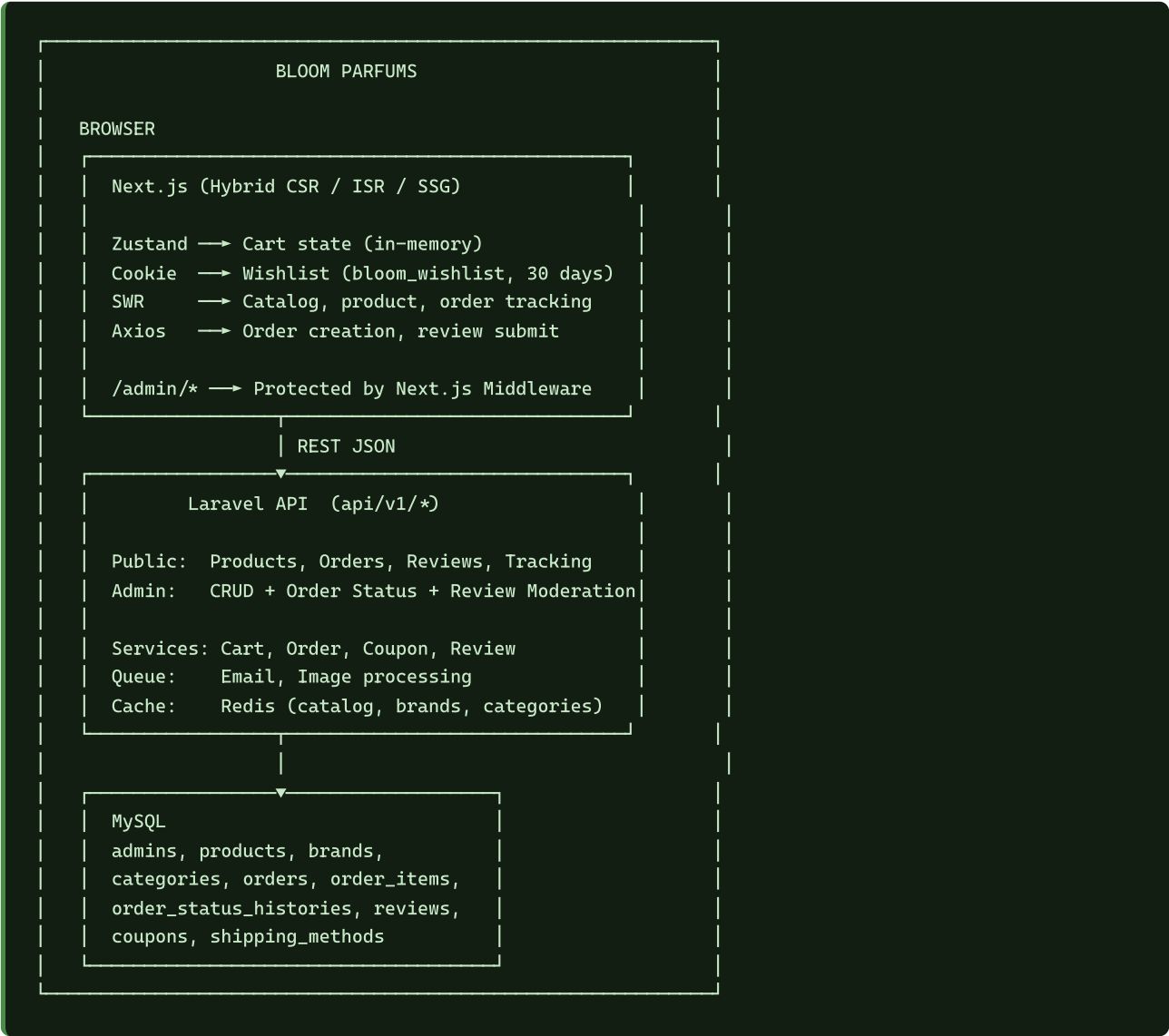
## 7. Final Verdict

### What This Project Should Be

**Bloom Parfums is a stateless-customer, admin-operated e-commerce storefront** with the following properties:

- Customers browse, add to cart, and check out **without creating an account**.
- The wishlist persists locally in a browser cookie — **zero backend involvement**.
- Orders are guest orders, trackable by `order_number` only.
- **One admin** manages the entire catalog, orders, and reviews through a protected backend panel.
- The frontend is a **Next.js hybrid**: SSG/ISR for catalog, CSR for interactive flows.
- The backend is a **thin Laravel API** — no views, no Blade, API-only.

Revised Architecture Summary



Development Sequence (Recommended)

| Phase   | Deliverables                                                              | Duration |
|---------|---------------------------------------------------------------------------|----------|
| Phase 1 | Laravel API: Products, Brands, Categories, Shipping Methods (public read) | Week 1   |
| Phase 2 | Next.js: Homepage, Collection, Product Detail wired to real API           | Week 1-2 |
| Phase 3 | Cookie wishlist system + Zustand cart + CartDrawer wired                  | Week 2   |
| Phase 4 | Laravel API: Order creation, guest tracking                               | Week 2-3 |
| Phase 5 | Next.js: Checkout → Success → Track → Status flow wired to real data      | Week 3   |
| Phase 6 | Review submission + admin review moderation                               | Week 3-4 |
| Phase 7 | Admin panel: login, product CRUD, order status management                 | Week 4-5 |
| Phase 8 | Production hardening: security headers, rate limiting, Redis, S3          | Week 5-6 |

---

*Final Technical Decisions Report | Bloom Parfums E-commerce Platform | February 2026*  
*This document supersedes ARCHITECTURE\_REPORT.md v1.0 for all implementation decisions.*

---

Bloom Parfums — Final Technical Decisions Report · February 25, 2026 · Supersedes v1.0 · Confidential